

Foundation:
Mental Models
& Surfaces

Scaling:
Subagents &
Isolation



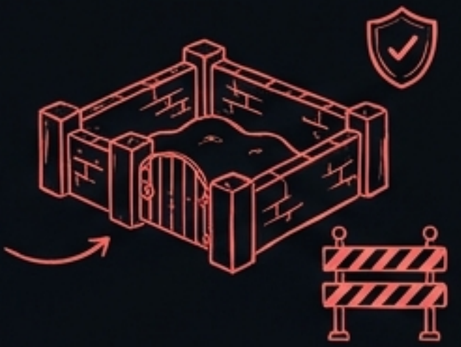
Architecting Agent-First Development on Google Cloud



Workflows:
Planning &
Artifacts

```
> plan agent_workflow
```

Security:
Sandboxes
& Guardrails





Behavior:
Rules, Skills
& Hooks

Lifecycle:
The Agents CLI
Production Loop

```
> agent deploy --prod
```



Antigravity is the developer harness, not the production runtime.

Developer Harness

Antigravity (App, CLI agy, IDE, SDK).
The coding agent you work with.

Agent-Building Expertise

Agents CLI skills + commands. Teaching the coding agent ADK, evals, and deployment.

Agent Framework

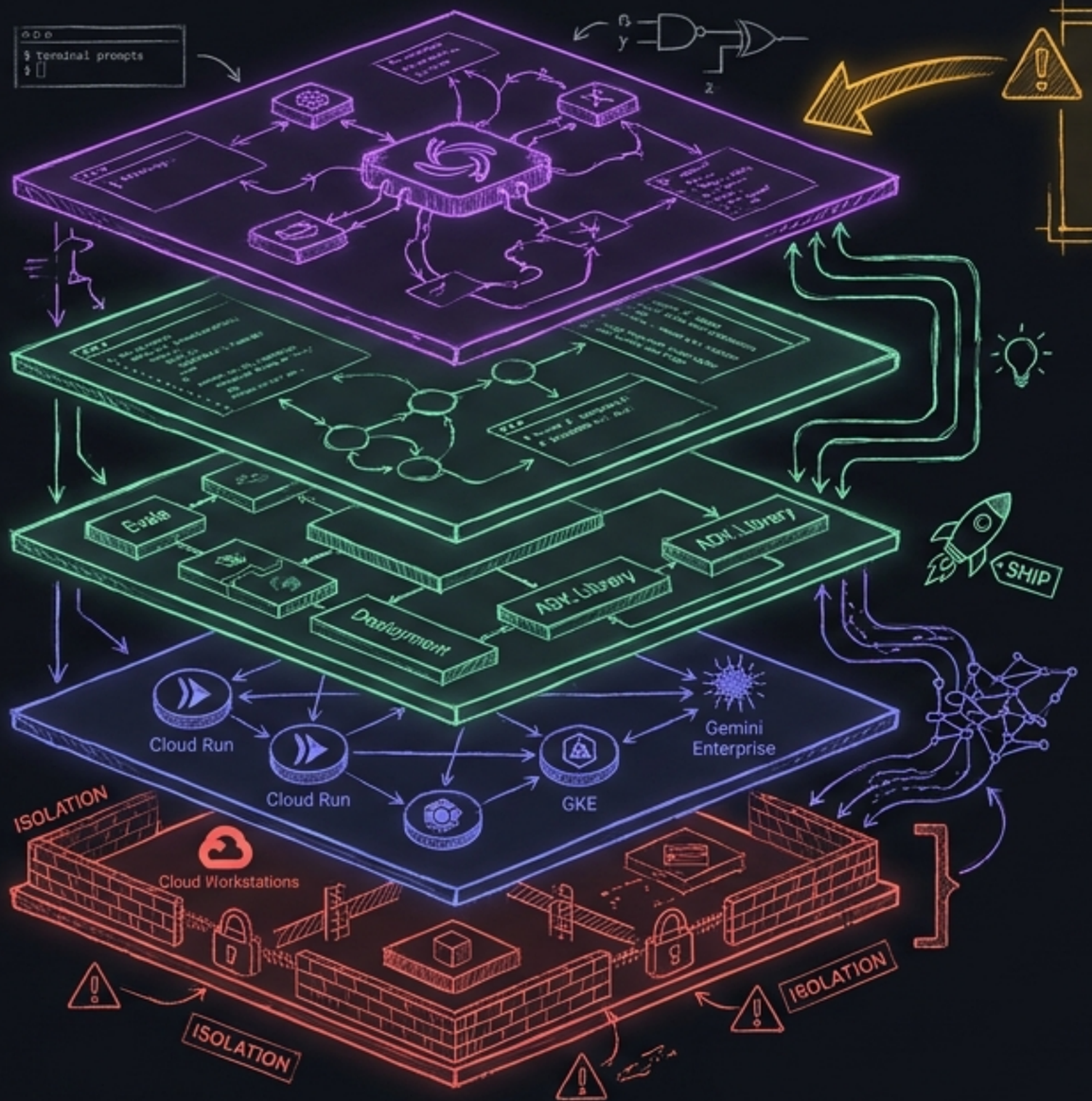
ADK (open source). The code of the agent you actually ship.

Production Runtime

Agent Runtime, Cloud Run, GKE -> Gemini Enterprise.
Where the shipped agent is distributed.

Secure Dev Environment

Cloud Workstations, Agent Sandbox on GKE, Terminal Sandbox. Where agent-run code is isolated.



Rule 0:
Never deploy Antigravity.
Deploy the ADK agent built by Antigravity.

RESTRICTED

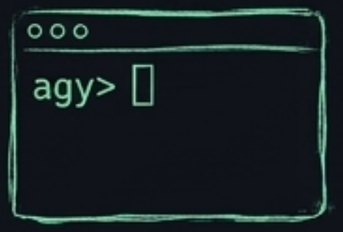
Four surfaces sharing one authenticated context.

Antigravity App v2.0



Standalone desktop app.
Best for running/monitoring multiple agents, Automation scheduling, and side-panel viewing of Subagents and Artifacts.

Antigravity CLI - agy



Terminal UI for remote/headless work.
Uses ~/.gemini/antigravity-cli/settings.json.
Type /remote-control to drive sessions from a browser.

Shared Sign-In context.

Antigravity IDE



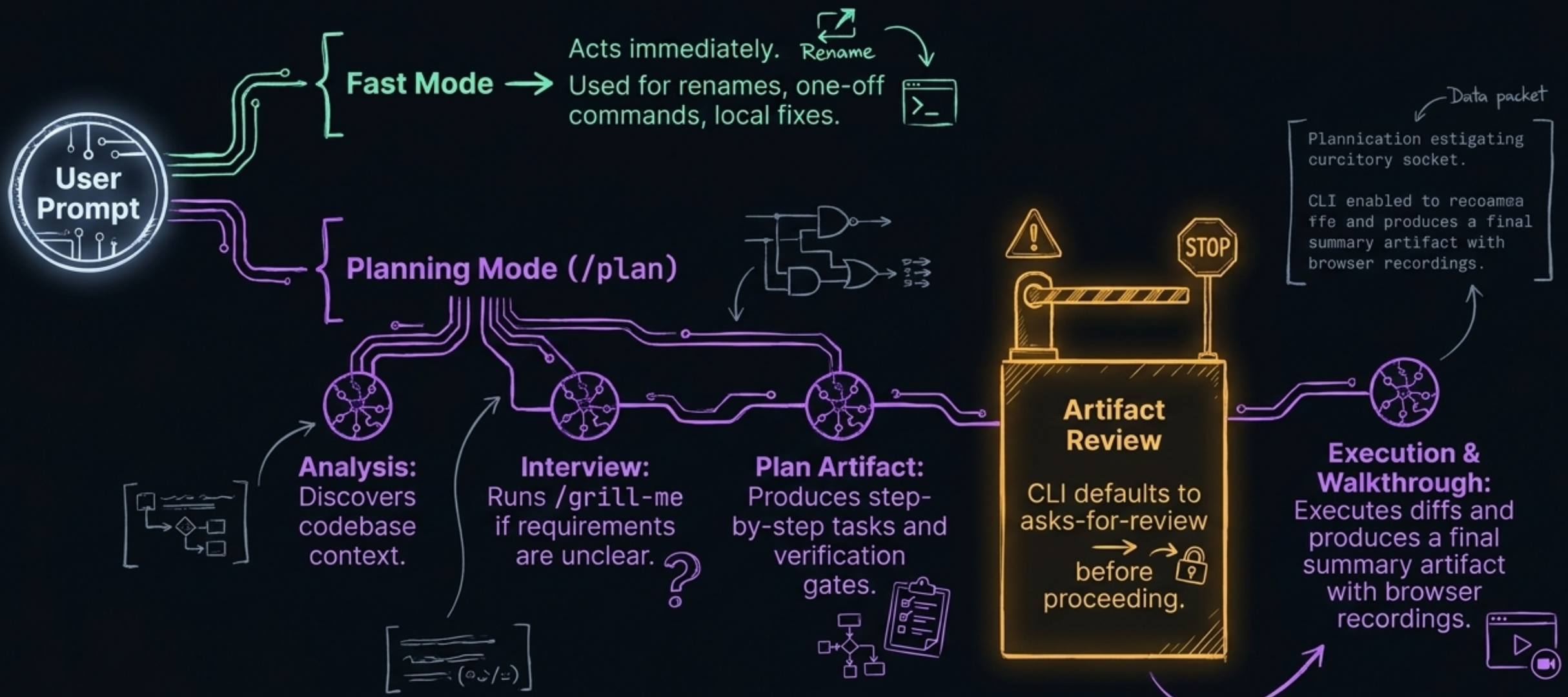
AI-first IDE built on VS Code.
Features Tab supercomplete and inline commands (Cmd/Ctrl+I).

Antigravity SDK



Python package (google-antigravity).
Read-only by default. Requires passing capabilities=CapabilitiesConfig() to enable write tools. Embeds agents in tests and pipelines.

Planning mode forces verification before execution.

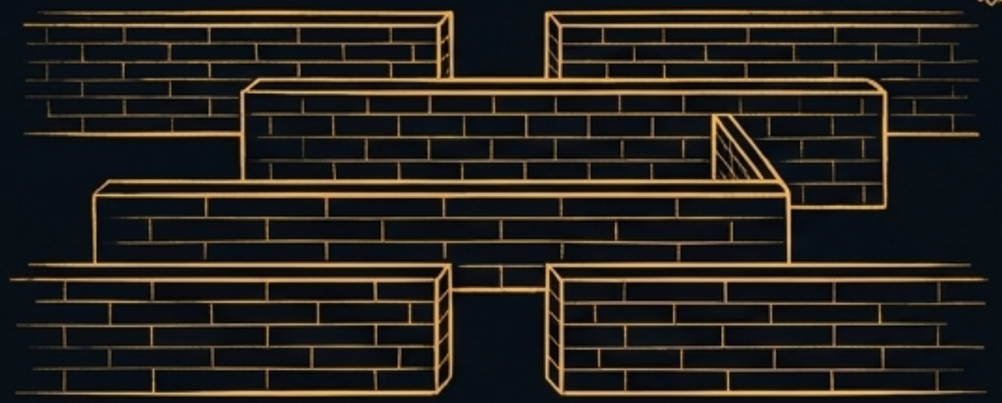


Differentiating execution playbooks by scope and proof.

	Multi-file Refactoring	Runtime Optimization	Vulnerability Patching
Goal/Scope	Change structure without altering external behavior across many modules.	Cut latency/cost on hot paths found in traces.	Narrowly remove one verified flaw at its root cause.
Proof	Unchanged test results + clean interface diff.	Better benchmark numbers + no regression in agents-cli eval compare.	The exploit PoC stops reproducing in an isolated sandbox.
Tooling	Branch worktrees, research subagents, .agents/rules/	Cloud Trace, Context Caching, Revision traffic split.	CodeMender (cm fix on Agent Platform) or manual security plan flow.

Shaping behavior with persistent rules and on-demand skills.

Rules

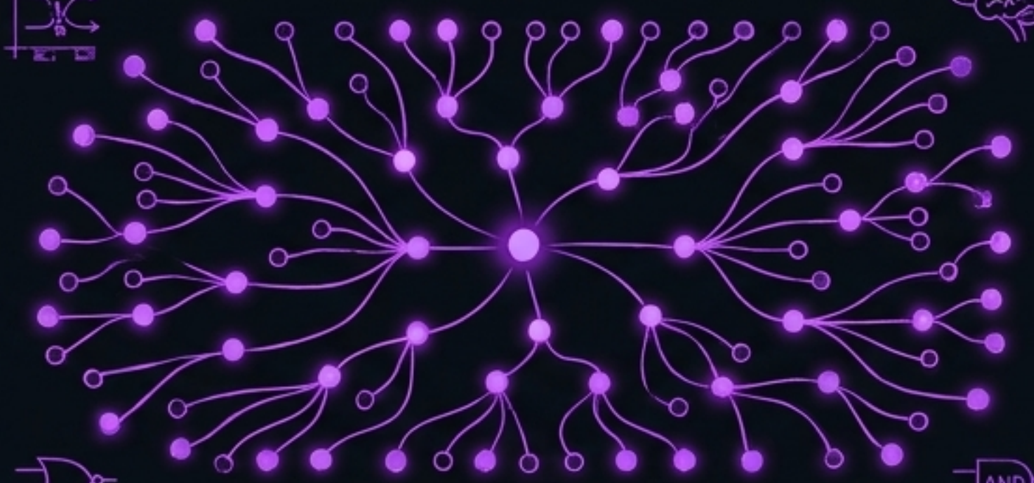


Persistent constraints and invariants (e.g., always use zod).

["Triggers": Require strict snake_case (always_on, model_decision, glob). CamelCase triggers cause silent drops.

["Limits": 24 KB per file limit. Shared 20,000-token budget. Over-budget rules are reduced to file-path pointers.

Skills



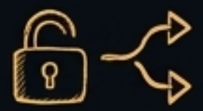
Multi-step workflows or runbooks.

["Structure": skills/<name>/SKILL.md (with scripts/ and references/ folders).

["Progressive Disclosure": Only the name and description sit in the context window. The description alone dictates whether the skill activates and loads the full body.

Intercepting and gating operations with synchronous hooks.

PreToolUse: Runs before a tool executes.
Returns: allow, deny, ask, force_ask (bypasses cache), or overwrite (shallow merge of arguments).



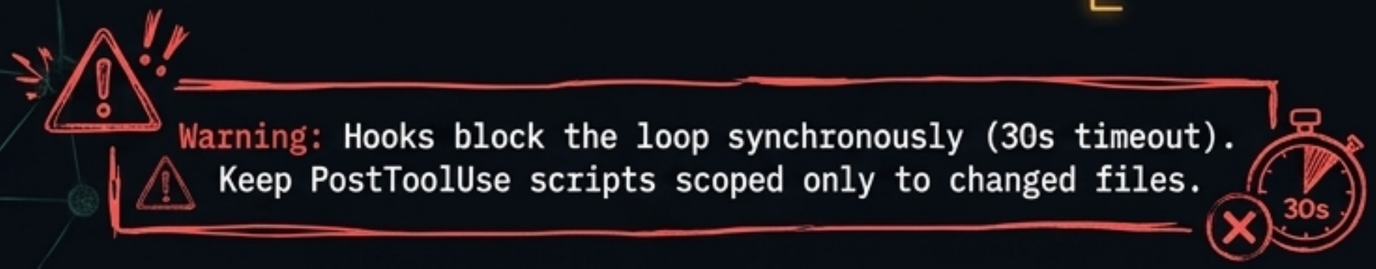


PostToolUse: Runs immediately after a tool finishes.
Use Case: Linting, formatting, analyzing changed files.
Returns an empty object (cannot steer the loop).

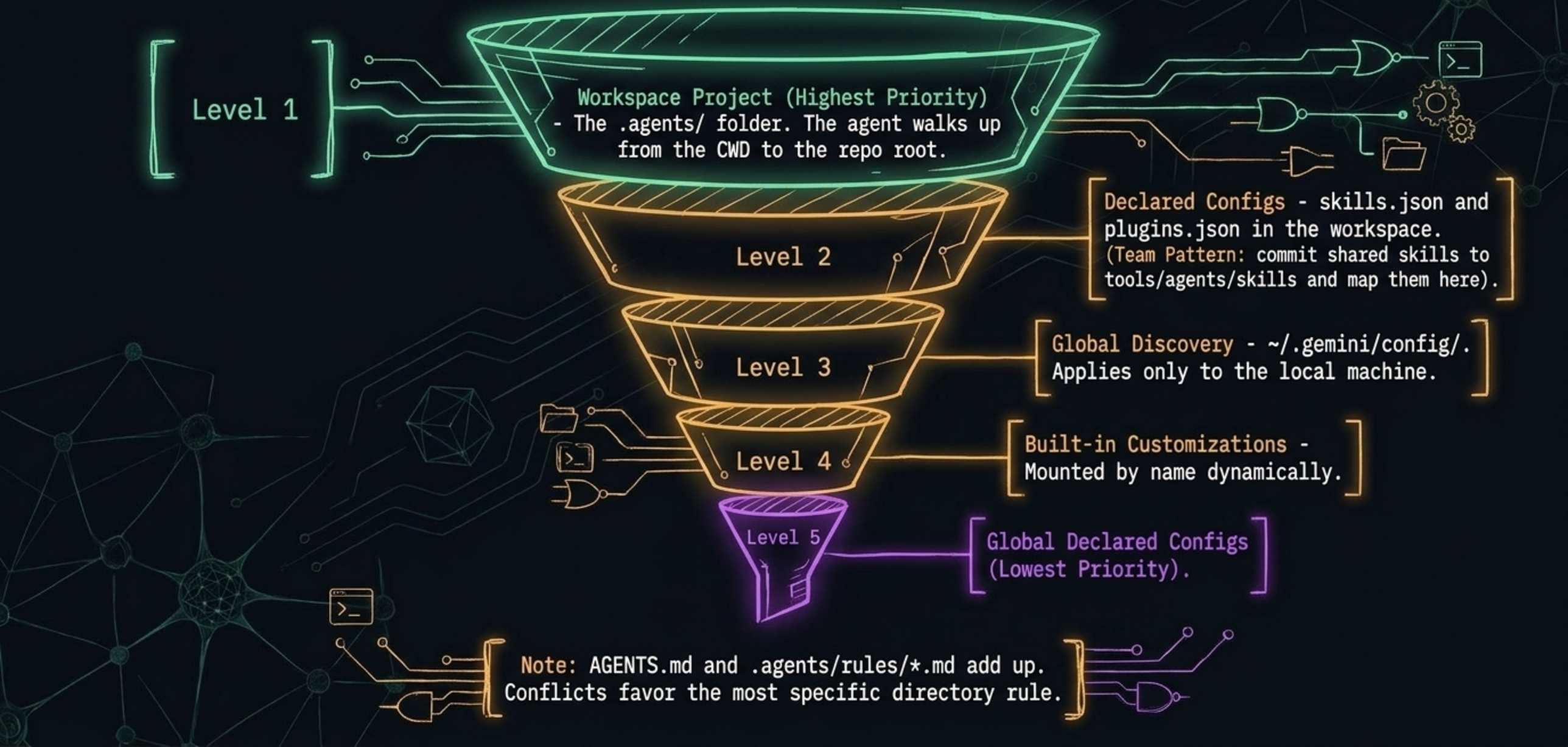


Stop: Fires when the loop attempts to end.
Use Case: The ultimate test gate.
Returns {'decision': 'continue', 'reason': '..'}. The reason injects directly into the context, forcing the agent to repair failing tests before exiting.





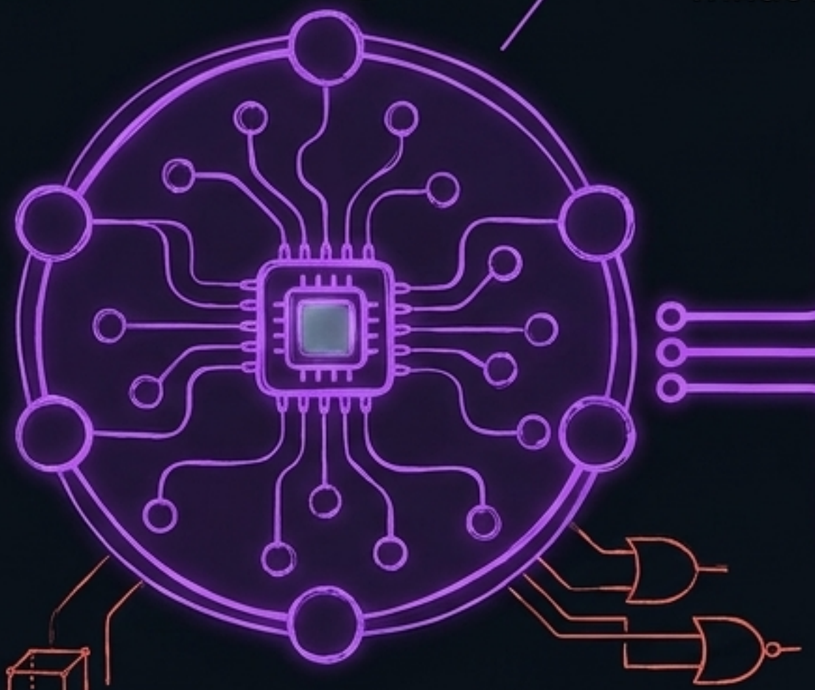
Discovery and priority cascade from the workspace down.



Orchestrating parallel work streams with specialized subagents

Main Agent

The Orchestration Mechanism: The main agent triggers `invoke_subagent`. Every subagent starts with a clean context window and runs asynchronously in the background.



- subagent_research
- subagent_browser
- subagent_self

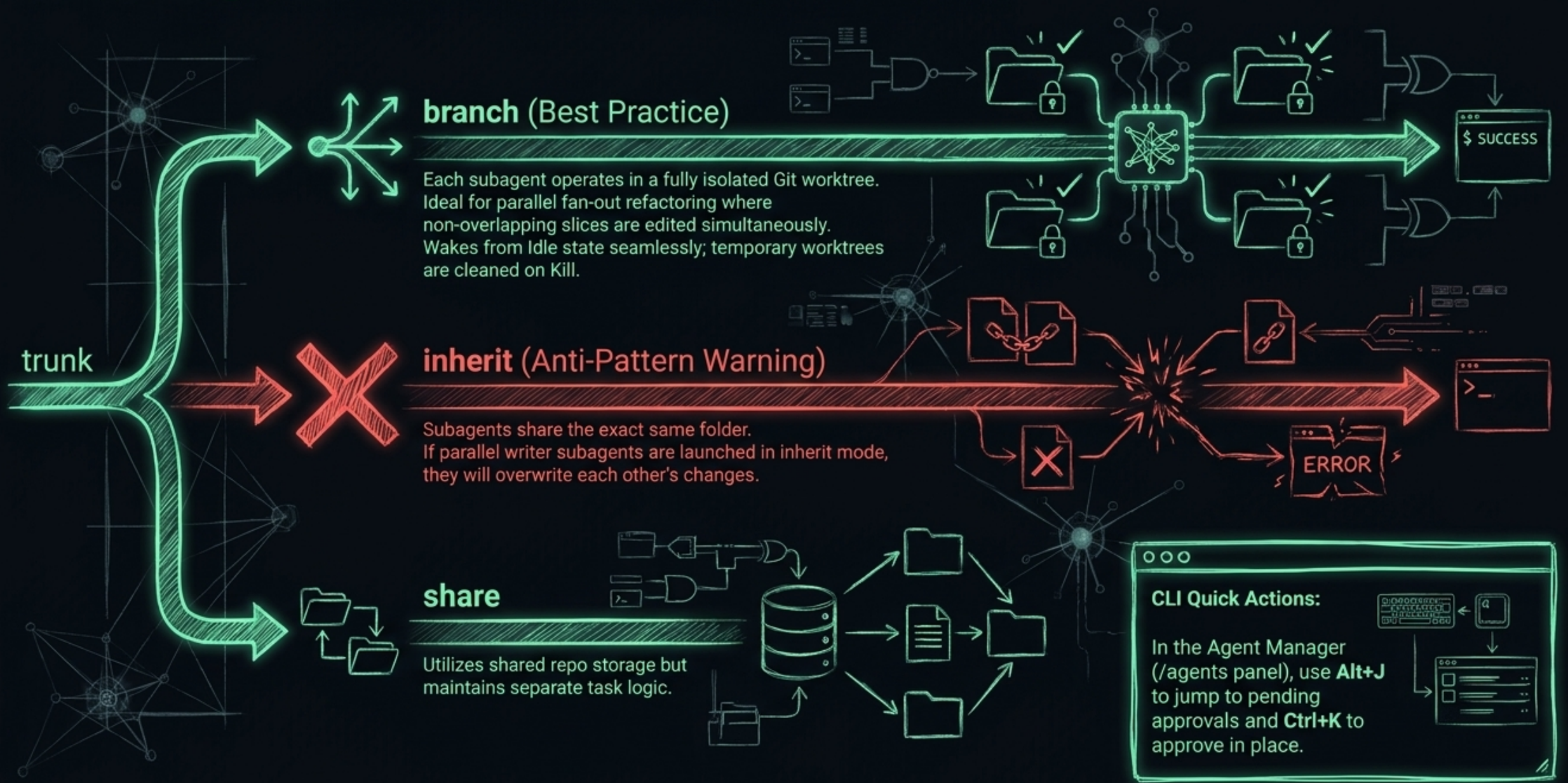
research: Codebase exploration and call-graph mapping.

browser: Drives Chrome via DevTools MCP. Fully isolated, records WebM artifacts, hidden from IDEs.

self: Dynamic cloning of the parent.

Custom Subagents: Defined in `.agents/agents/<name>.md`. YAML frontmatter dictates `tools`, `model` (`inherit`/`flash`/`pro`), and MCP servers.

Preventing collisions with Git worktree isolation.



The Deny > Ask > Allow local permission hierarchy.



The Hierarchy:
Default behavior denies out-of-workspace operations.
Rules use typed matchers (command, write_file, mcp).
Network defaults to blocking non-allowed domains.

default:
Requests diff review before file writes.

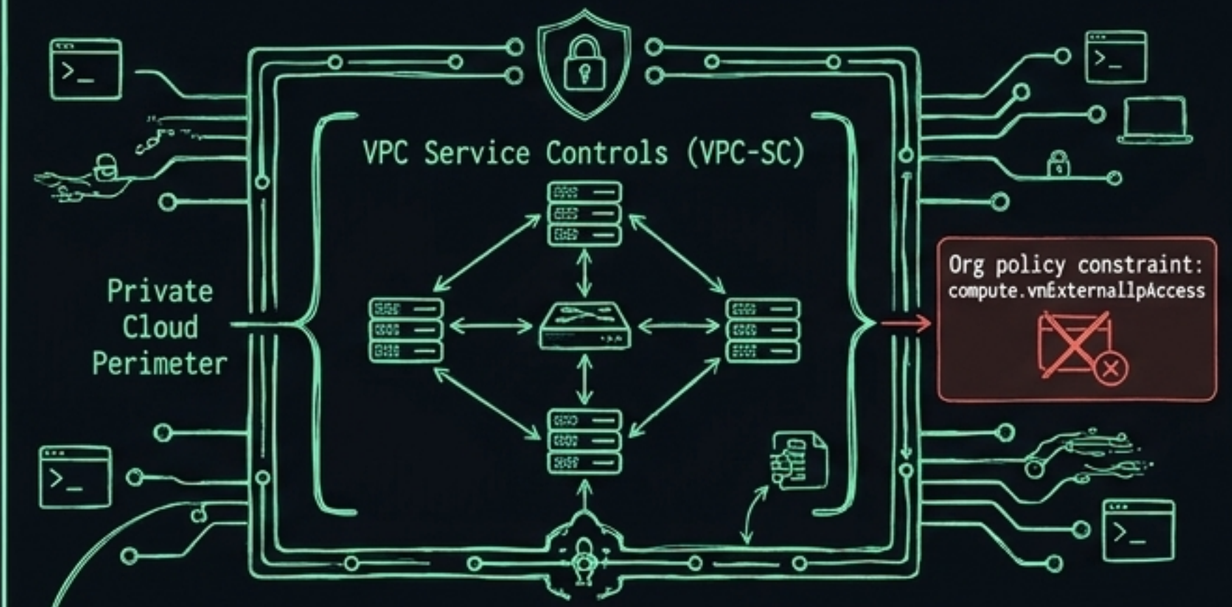
accept-edits:
Auto-approves workspace edits; commands follow toolPermission.

auto (Preview):
Auto-approves edits AND runs commands inside the sandbox. Reversible commands outside the sandbox run automatically; risky ones pause.

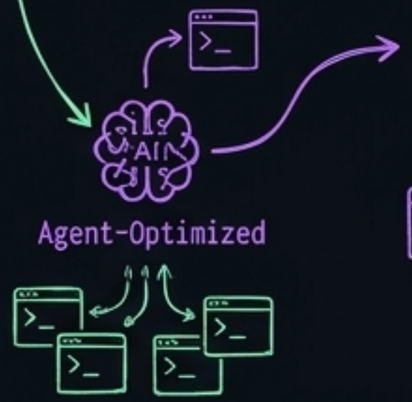
[The Terminal Sandbox Constraint: Project folders mount read-write; read_file paths mount read-only. Blocked files (like .env) are are strictly invisible via macOS/Linux kernel namespaces.]

Enterprise isolation: Cloud Workstations vs. GKE Sandboxes

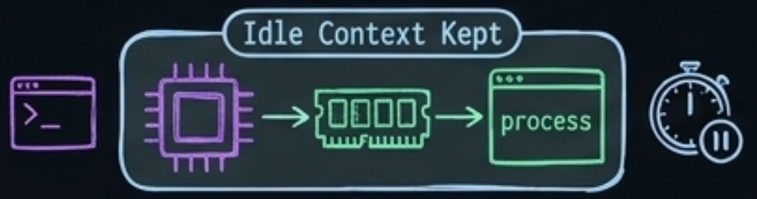
Cloud Workstations (For Human & Agent Collaboration)



Architecture: Private clusters behind VPC Service Controls (VPC-SC) with no public IPs (org policy constraint: compute.vmExternalIpAccess)

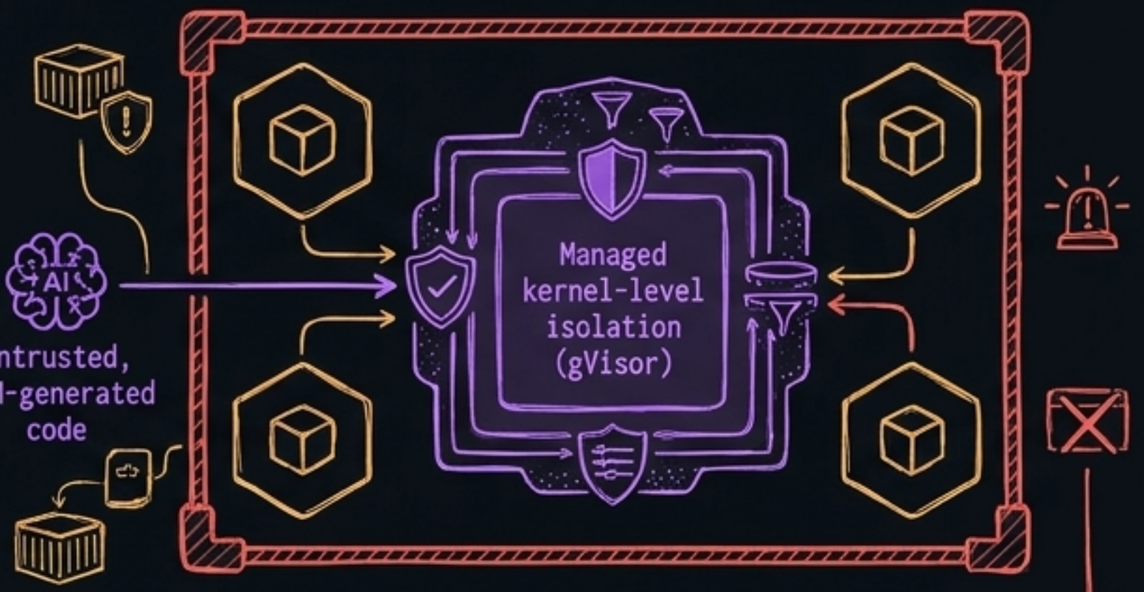


Configure idleAction=SUSPEND with keep-alive lifecycle hooks (/google/scripts/keep_alive.sh)

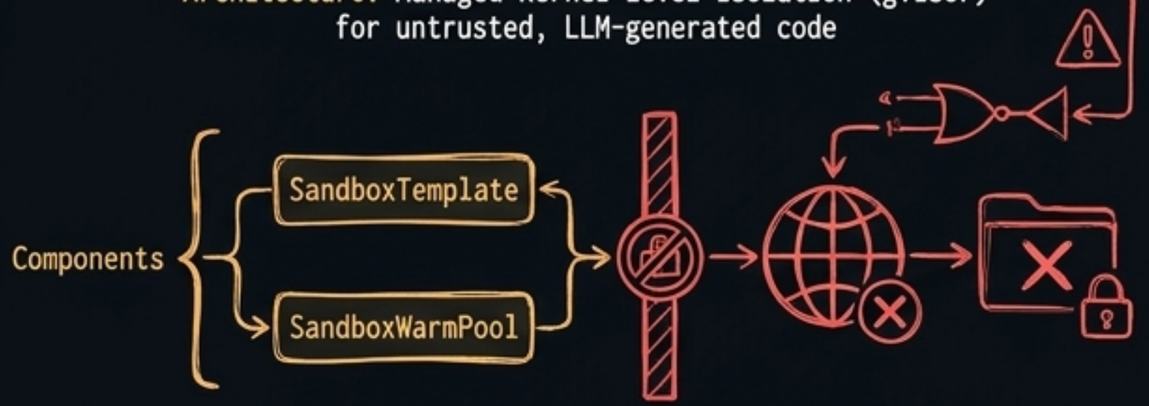


Keeps context, RAM, and processes intact without paying for idle vCPU

Agent Sandbox on GKE (For Automated Scaling)

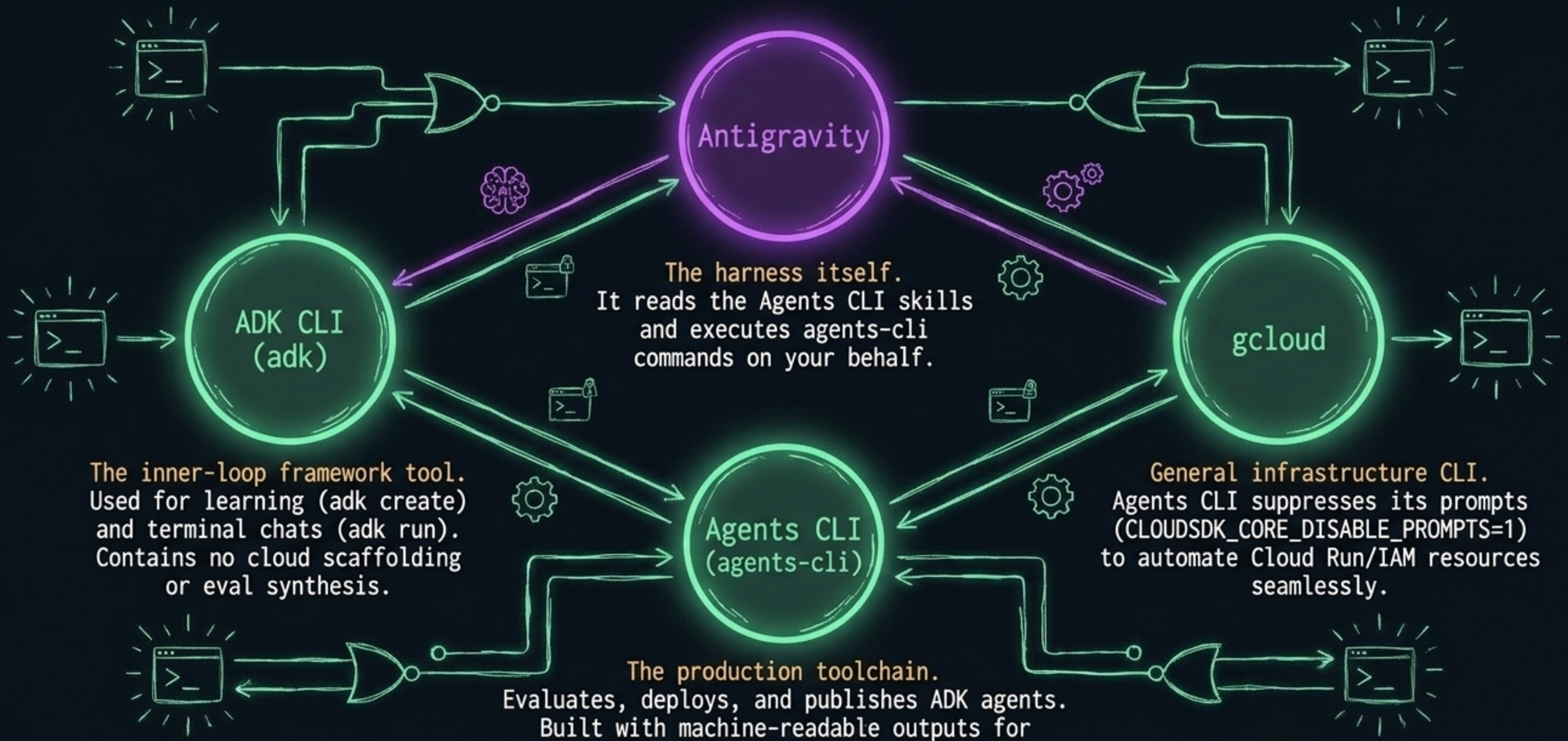


Architecture: Managed kernel-level isolation (gVisor) for untrusted, LLM-generated code



Requires SandboxTemplate and SandboxWarmPool.
Security policies strictly block hostNetwork and hostPath.

Disambiguating the agent-first tooling ecosystem.



Injecting build and deployment expertise via Bundled Skills

```
$ agents-cli setup --workspace
```

(installs skills relative to the CWD so teammates inherit them via Git).

workflow: The master end-to-end flow and model selection rules.

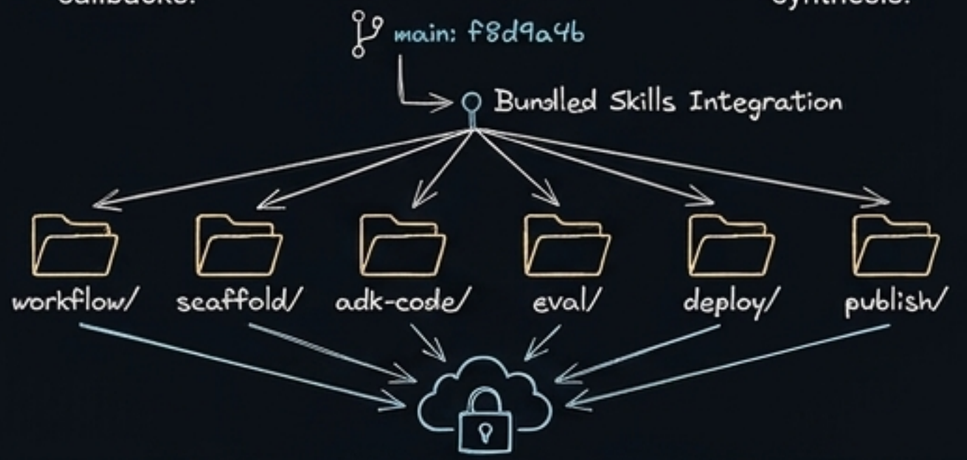
scaffold: Handles -p prototype flags, deployment targets, and CI/CD generation.

adk-code: Teaches the agent ADK APIs, orchestration, and callbacks.

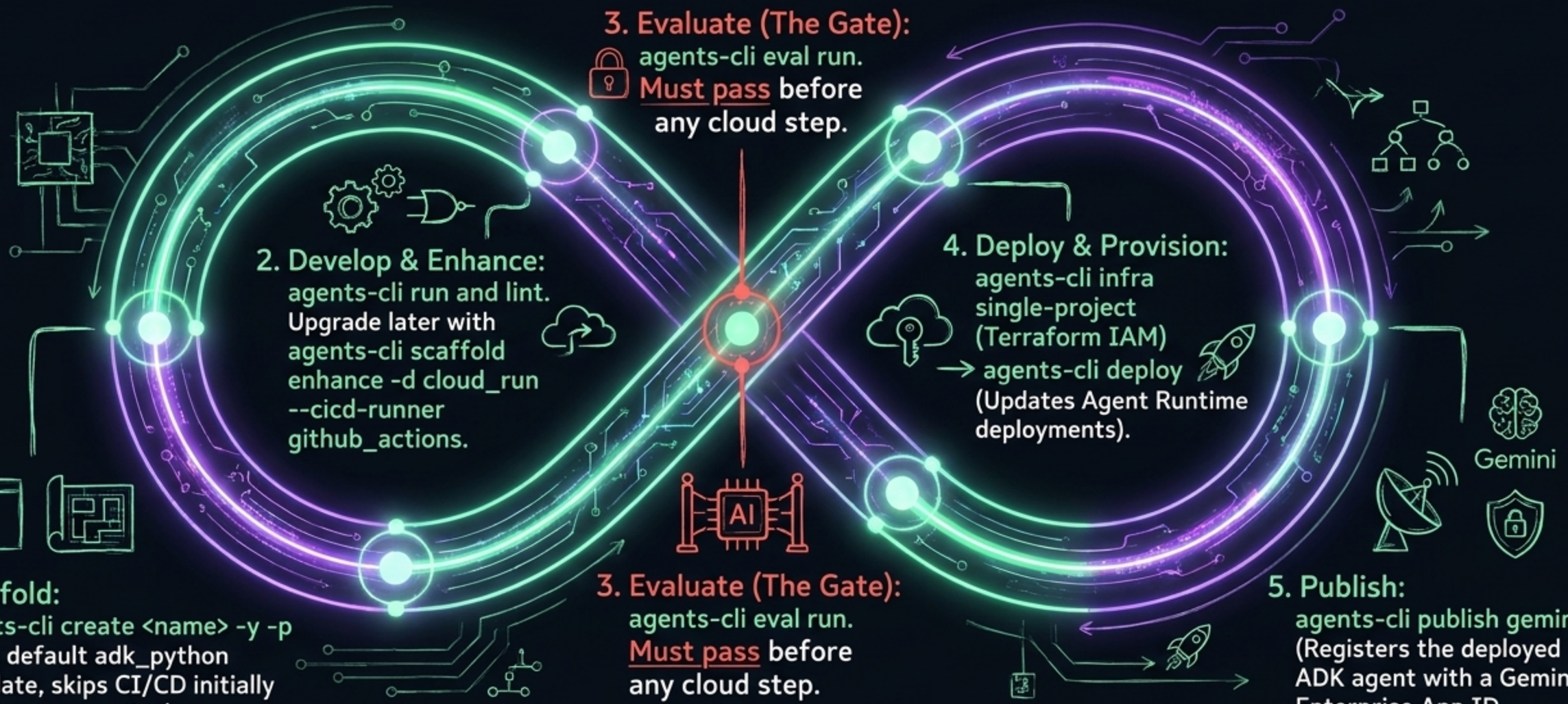
eval: Drives LLM-as-judge, failure clustering, and dataset synthesis.

deploy: Handles Agent Runtime, Cloud Run, secrets, and rollbacks.

publish: Manages Gemini Enterprise registration (ADK vs. A2A).



The automated build, evaluate, deploy, and publish loop.



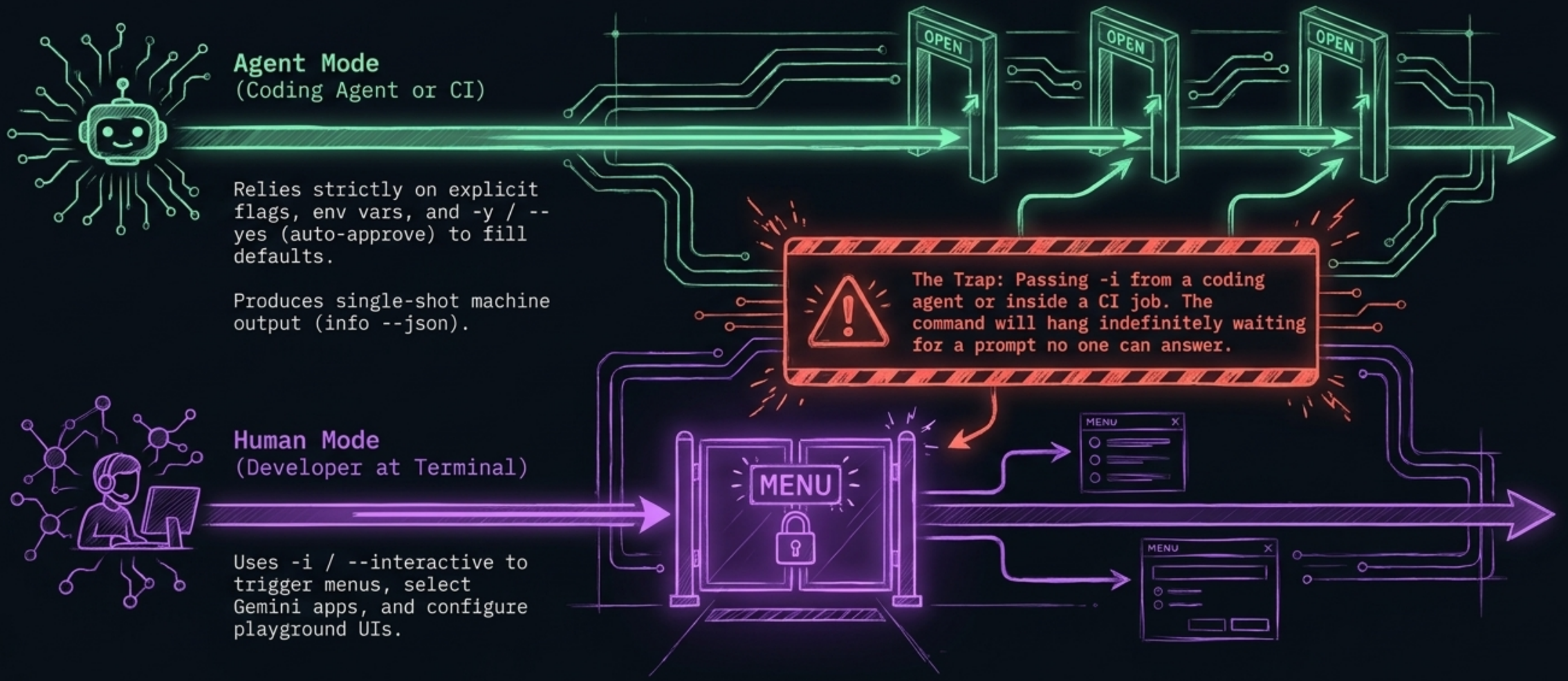
1. Scaffold:
agents-cli create <name> -y -p
(Uses default adk_python template, skips CI/CD initially for fast prototyping).

3. Evaluate (The Gate):
agents-cli eval run.
Must pass before any cloud step.

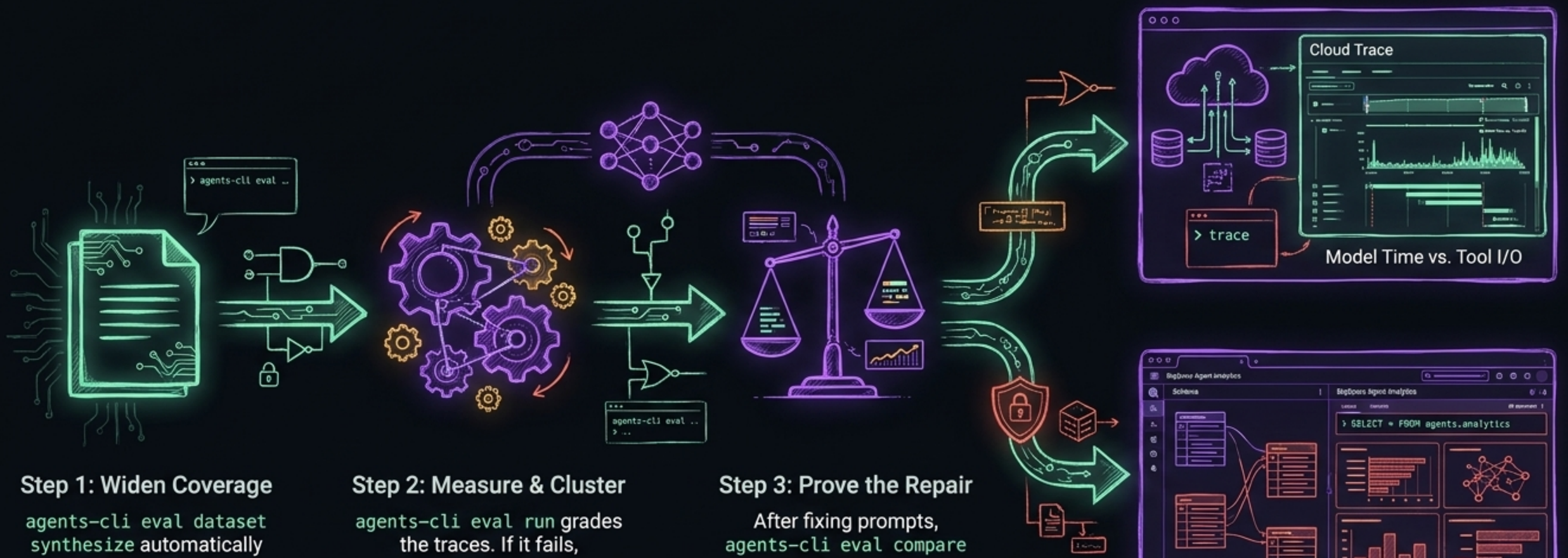
5. Publish:
agents-cli publish gemini
(Registers the deployed ADK agent with a Gemini Enterprise App ID securely via VPC-SC).

Input mechanics: Agent Mode versus Human Mode.

(Note: There is no --mode agent flag. Behavior is driven entirely by input style.)



Synthesizing, gating, and observing agent quality



Step 1: Widen Coverage

`agents-cli eval dataset synthesize` automatically generates multi-turn eval scenarios around reported failures

Step 2: Measure & Cluster

`agents-cli eval run` grades the traces. If it fails, `agents-cli eval analyze` clusters failures by root cause.

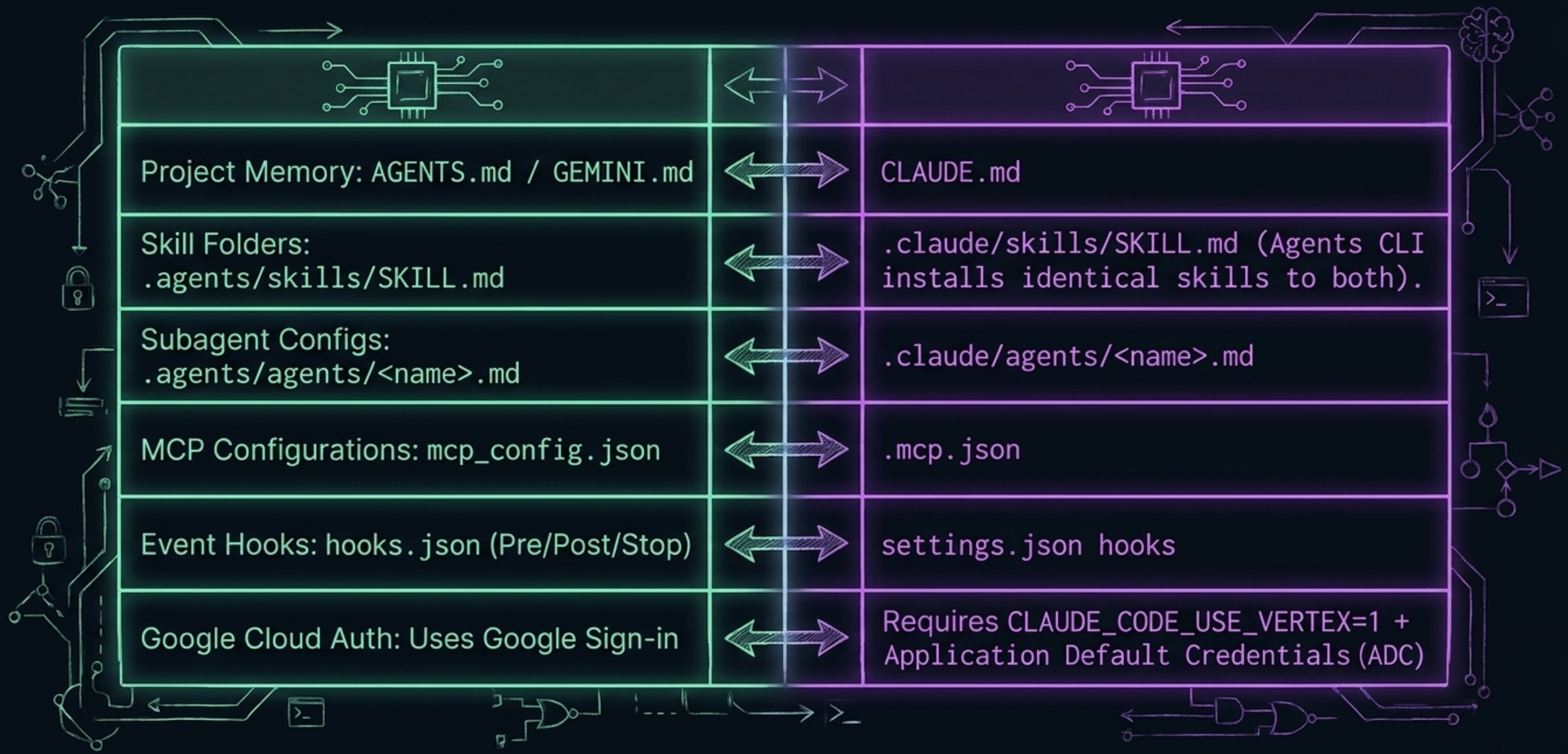
Step 3: Prove the Repair

After fixing prompts, `agents-cli eval compare baseline.json candidate.json` proves mathematically that no regressions occurred.

Step 4: Cloud Observability

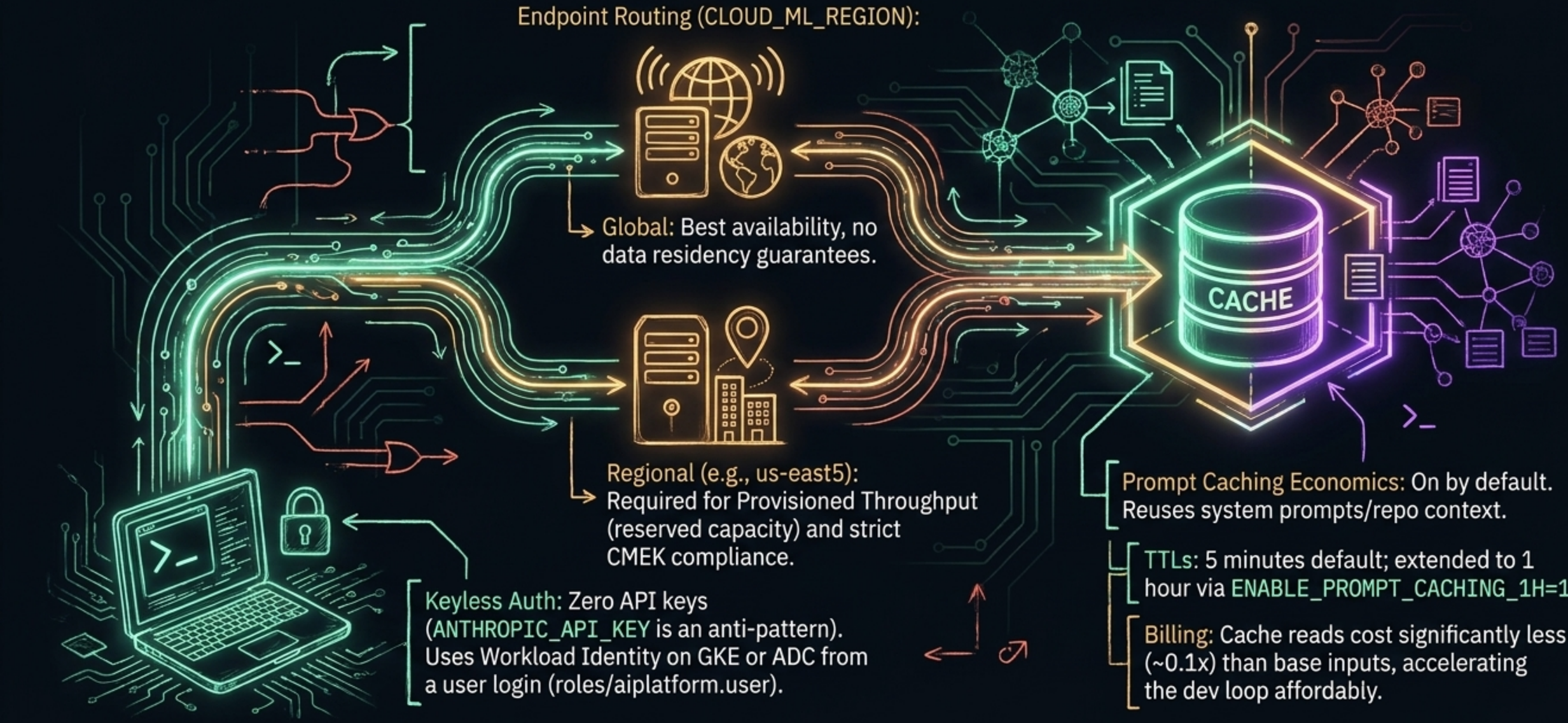
Once deployed, telemetry flows automatically to Cloud Trace (separating model time from tool I/O) and BigQuery Agent Analytics.

Translating the ecosystem: Antigravity vs. Claude Code.

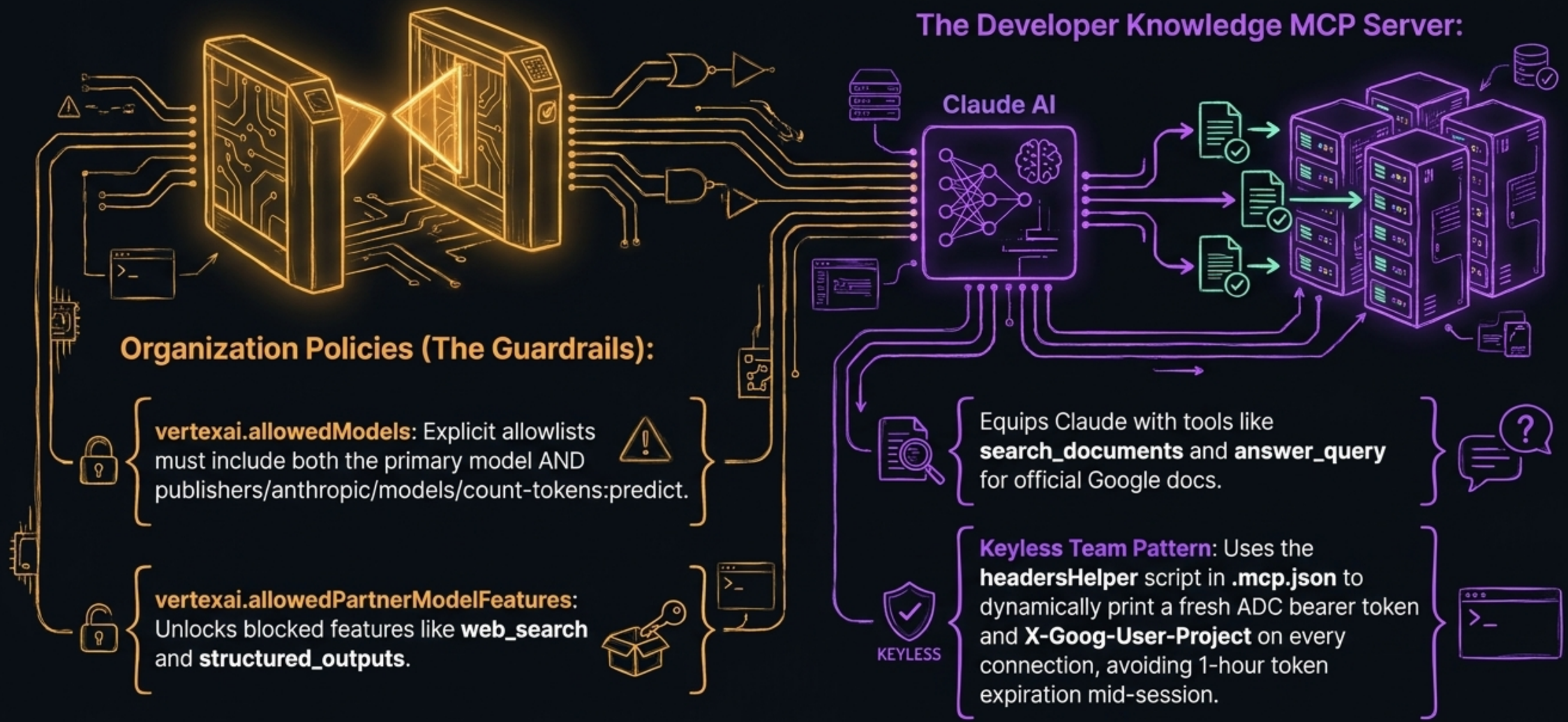


Project Memory: `AGENTS.md / GEMINI.md`	⟷	`CLAUDE.md`
Skill Folders: `.agents/skills/SKILL.md`	⟷	`.claude/skills/SKILL.md` (Agents CLI installs identical skills to both).
Subagent Configs: `.agents/agents/<name>.md`	⟷	`.claude/agents/<name>.md`
MCP Configurations: `mcp_config.json`	⟷	`.mcp.json`
Event Hooks: `hooks.json` (Pre/Post/Stop)	⟷	`settings.json hooks`
Google Cloud Auth: Uses Google Sign-in	⟷	Requires `CLAUDE_CODE_USE_VERTEX=1` + Application Default Credentials (ADC)

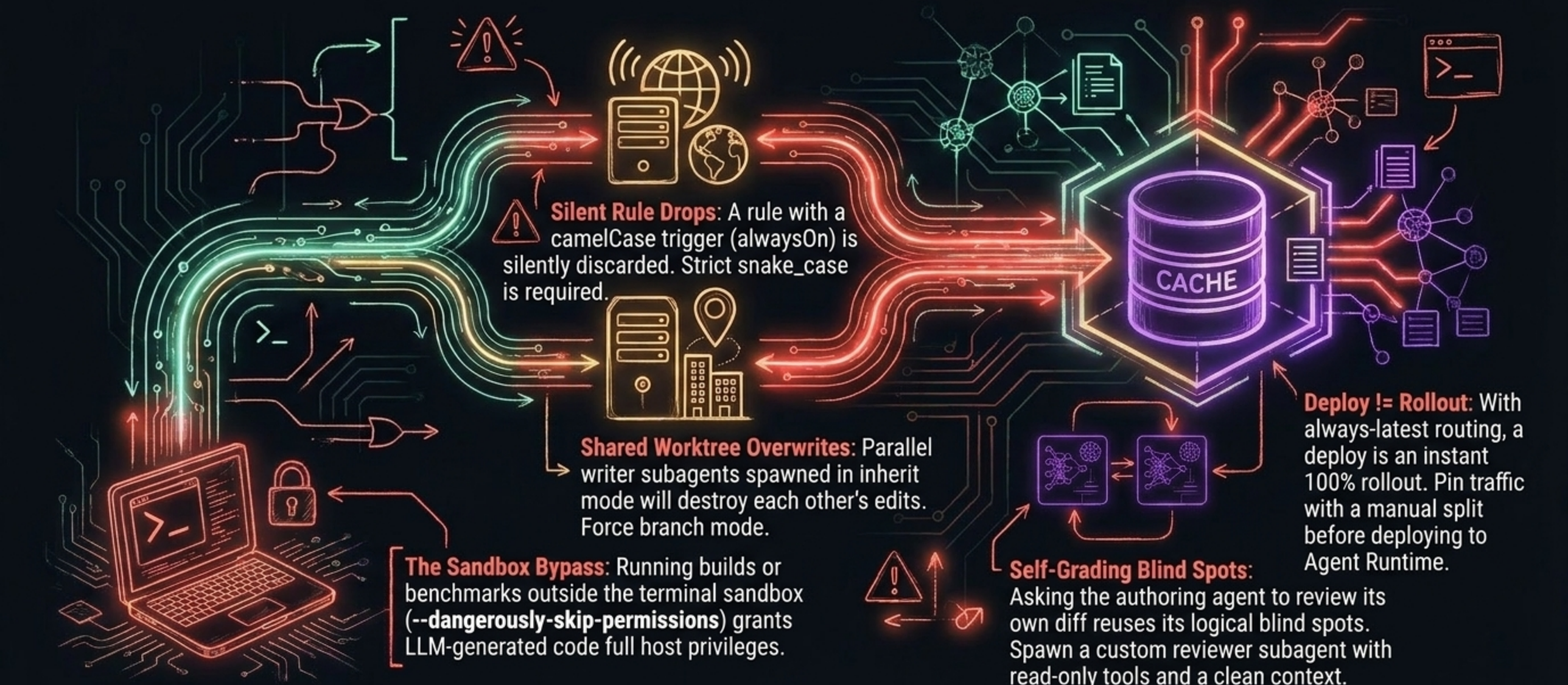
Deploying Claude Code on Agent Platform.



Enforcing guardrails and grounding developer knowledge.



System mastery: Identifying architecture anti-patterns.



Silent Rule Drops: A rule with a camelCase trigger (alwaysOn) is silently discarded. Strict snake_case is required.

Shared Worktree Overwrites: Parallel writer subagents spawned in inherit mode will destroy each other's edits. Force branch mode.

The Sandbox Bypass: Running builds or benchmarks outside the terminal sandbox (**--dangerously-skip-permissions**) grants LLM-generated code full host privileges.

Self-Grading Blind Spots: Asking the authoring agent to review its own diff reuses its logical blind spots. Spawn a custom reviewer subagent with read-only tools and a clean context.

Deploy != Rollout: With always-latest routing, a deploy is an instant 100% rollout. Pin traffic with a manual split before deploying to Agent Runtime.